
Machine Learning A

2026-2027

Home Assignment 5

Christian Igel

Sadegh Talebi

Yevgeny Seldin

Department of Computer Science
University of Copenhagen

The deadline for this assignment is **8 October 2026, 22:00**. You must submit your *individual* solution electronically via the Absalon home page.

A solution consists of:

- A PDF file with detailed answers to the questions, which may include graphs and tables if needed. Do *not* include your full source code in the PDF file. You may include short, selected code snippets if you are asked to do so or if they help improve the readability or clarity of your answers.
- Your solution source code, with comments about the major steps involved in each question (see below). If your solution consists of multiple source files, submit them together in a .zip file; if there is only one source file, it may be submitted directly without zipping. Source code must be submitted in the original file format, not as PDF. The programming language of the course is Python.
- **IMPORTANT: Do NOT zip the PDF file**, since zipped files cannot be opened in *SpeedGrader*. Zipped PDF submissions will not be graded.
- Your PDF report should be self-sufficient. That is, it should be possible to grade your submission without opening the source-code submission. In particular, all requested outputs, such as plots, tables, and numerical values, must be generated and included in the PDF report. We do not guarantee that the source-code submission will be opened during grading.
- Your code should be structured such that there is one main file (or one main file per question) that we can run to reproduce all the results presented in your report. This main file can, if you like, call other files with functions, classes, etc.

- Handwritten solutions will not be accepted. Please use the provided LaTeX template to write your report.
- You must adhere to the notations used in the task and in the lectures. If you need to use additional notations, ensure they are clearly and precisely defined. Undefined notations will result in point deduction.

1 On the Role of Dependence (8 points) [Sadegh, W5 Monday]

Design an example of identically distributed, but *dependent* Bernoulli random variables $X_1, \dots, X_n$ (i.e., $X_i \in \{0, 1\}$), such that

$$\mathbb{P}\left(\left|\mu - \frac{1}{n} \sum_{i=1}^n X_i\right| \geq \frac{1}{2}\right) = 1,$$

where $\mu = \mathbb{E}[X_1]$.

Note that in this case $\frac{1}{n} \sum_{i=1}^n X_i$ does not converge to μ as n goes to infinity. The example shows that independence is crucial for convergence of empirical means to the expected values.

2 On Confidence Intervals (12 points) [Sadegh, W5 Monday]

We are interested in estimating the mean of a random variable, denoted by θ . Using n observations and a certain method, we have constructed the following 98%-CI:

$$I = [0.237, 0.439].$$

2.1 Part 1

Determine whether each of the following statements is **True** or **False**. Provide a brief justification (1-2 lines) for your answer.

1. All the n observations used to construct I belong to I .
2. 98% of the n observations used to construct I belong to I .

3. Suppose another set of n observations is used to construct a new 98%-CI using *the same method*. The resulting confidence interval will necessarily be equal to I .
4. Using the same observations, we construct two CIs: a 98%-CI, denoted by J , and a 99%-CI, denoted by J' (see the slides for the Hoeffding-based confidence interval). It is necessarily true that $J \subseteq J'$.

2.2 Part 2

Consider again $I = [0.237, 0.439]$. Suppose your boss wants the confidence interval to be reported to two decimal places, while preserving *at least the same confidence guarantee*. Report the tightest such interval. What interval would you report? Briefly justify your answer.

3 Loss Range Correction in Generalization Bounds (10 points) [Sadegh, W5 Monday]

Suppose your former colleague assessed an ML algorithm and derived a generalization bound for a classifier h using a validation sample S of size 1000. Using Hoeffding's inequality, she reported the following bound:

$$L(h) \leq 0.38, \quad \text{with probability at least } 0.95.$$

In her analysis, she assumed that the loss function ℓ takes values in $[0, 1]$. However, it turns out that the correct range of the loss is $[0.18, 1.44]$.

- (i) Does the reported bound remain valid? Briefly explain why or why not.
- (ii) Assume that the validation sample S is no longer available. Can the new information about the range of the loss still be incorporated to obtain a corrected generalization bound? If so, derive the new bound. If so, derive the corrected bound and report it to three decimal places (i.e., $L(h) \leq 0.\square\square\square$).

4 Convolutional Neural Networks (70 points)

[Christian, W5 Friday]

4.1 Sobel filter (24 points) [Christian, W5 Friday]

In this assignment, you should implement a Sobel filter in PyTorch. The learning goals are a better understanding of basic image filtering operations, a deep understanding of how the PyTorch convolutional layers work, and better PyTorch programming skills.

The Sobel filter is a basic operator to highlight edges in images (e.g., see Wikipedia). It convolves the image with two 3×3 filters, the results of which are combined. Let the matrix $\mathbf{I}$ represent a (single-channel) 2D input image and $*$ the convolution operator. Then the Sobel filter computes in a first step

$$\mathbf{G}_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} * \mathbf{I} \quad \text{and} \quad \mathbf{G}_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{I} , \quad (1)$$

which are then combined to a feature map

$$[\mathbf{G}]_{ij} = \sqrt{[\mathbf{G}_x]_{ij}^2 + [\mathbf{G}_y]_{ij}^2} , \quad (2)$$

where $[\cdot]_{ij}$ denotes the matrix element at position ij .

Start by going through the notebook `PyTorch 2D convolutions.ipynb`, which gives some examples on how to work with 2D convolutions in PyTorch and provides some basics on how to read and display images. Do this carefully. First, it is surprising how many people doing deep learning in Python actually do not know what is going in a simple convolutional layer. Second, handling input images in real-world applications can sometimes be tricky (and I am not claiming that the way I did it in the notebook is the most elegant one).

Implement the Sobel filter. Use a single `nn.Conv2d` call to generate the two feature maps $\mathbf{G}_x$ and $\mathbf{G}_y$. For squaring each element in a filter output you can use something like `torch.square`. For combining filter outputs as in (2) you can use `torch.sum`, but you have to set the `axis` argument properly.

Deliverables: Show the important parts of your code (i.e., the Sobel filter) in the report. Apply the Sobel filter to the same input image as in `PyTorch 2D convolutions.ipynb`.

Deliverables: Show plots of the two feature maps and the final result (i.e., $\mathbf{G}_x$, $\mathbf{G}_y$, and $\mathbf{G}$).

Deliverables: In the report, discuss how you considered the difference between convolution and cross-correlation (note that the latter is what the “convolutional” layer is doing) in your implementation.

4.2 Convolutional neural networks (24 points)

[Christian, W5 Friday]

The learning goal of this part of the assignment is to get more comfortable with implementing convolutional neural networks (CNNs) in PyTorch.



Figure 1: Examples from the traffic sign data set.

Consider the notebook `Torch_Traffic_Signs_Basic_Template.ipynb` and the file `GTSRBTrafficSigns.py`. The notebook is a template for training a convolutional neural network (CNN) for classifying traffic signs on the same data as the competition described by Stallkamp et al. (2012). The data is handled by the `Dataset` class in the file `GTSRBTrafficSigns.py`. Recognition of traffic signs is a challenging real-world problem of high industrial relevance. Traffic sign recognition can be viewed as a multi-class classification problem with unbalanced class frequencies, in which one has to cope with large variations in visual appearances due to illumination changes, partial occlusions, rotations, weather conditions, etc. However, humans are capable of recognizing the large variety of existing road signs with close to 100 % correctness – not only in real-world driving situations, which provides both context and multiple views of a single traffic sign, but also when looking at single images as those shown in Figure 1. Now the question is how good a computer can become at solving this problem.

The notebook is supposed to run with GPU support, for example, using *Google Colaboratory*. Even then, executing it will take some time.

The notebook misses the CNN definition. Define a network with a

- convolutional layer creating 64 feature maps using 5×5 kernels followed by a
- 2×2 max-pooling layer followed by a
- convolutional layer creating 64 feature maps using 5×5 kernels followed by a
- 2×2 max-pooling layer followed by a
- linear fully-connected layer with 43 outputs.

After each convolutional layer, the ELU (exponential linear unit, Clevert et al. (2016)) activation function should be applied.

Deliverables: Show the model definition in the report.

You should get the following output when you print your model:

```
Net(  
  (conv1): Conv2d(3, 64, kernel_size=(5, 5), stride=(1, 1))  
  (pool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation  
    =1, ceil_mode=False)  
  (conv2): Conv2d(64, 64, kernel_size=(5, 5), stride=(1, 1))  
  (pool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation  
    =1, ceil_mode=False)  
  (fc2): Linear(in_features=1024, out_features=43, bias=True)  
)
```

4.3 Augmentation (22 points) [Christian, W5 Friday]

Convolutional neural networks are highly complex models. To reduce the risk of overfitting and to be able to learn difficult tasks with a high input variability, many training examples are needed. *Data augmentation* is a way to enlarge the training data set by artificial training points generated from the available data. Data augmentation refers to applying transformations to the input data (the images) that do not change their labels. If we know, for example, that the classifications of images do not change when the images are rotated, then showing rotated versions of the images during the training process helps to learn this invariance property. In the context of neural networks, this type of data augmentation can be traced back to Baird (1992). It has been used successfully for CNNs, for instance, already in the influential work by Krizhevsky et al. (2012). The learning goal of this part of the assignment is to get more experience in using data augmentation.

Inspect the Data class in `GTSRBTrafficSigns.py`.

Deliverables: Answer the following questions briefly in your report: Which transformations are applied to the input images during training? Why is a transformation conditioned on the label? Please add at least one additional (not completely nonsensical) transformation.

Deliverables: Show the corresponding code in your report and briefly argue why you think that this may be a reasonable augmentation given the task.

References

- Henry S. Baird. Document image defect models. In *Structured Document Image Analysis*, pages 546–556. Springer, 1992.
- Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). In Yoshua Bengio and Yann LeCun, editors, *International Conference on Learning Representations (ICLR)*, 2016.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, 2012.
- Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32:323–332, 2012.